`threads_in_pop` to be equal to 1 and the attempt to delete the nodes, the more chance there is that another thread has called `pop()`, and that `threads_in_pop` is no longer equal to 1, thus preventing the nodes from actually being deleted.

In high-load situations, there may *never* be such a quiescent state, because other threads have entered `pop()` before all the threads initially in `pop()` have left. Under such a scenario, the `to_be_deleted` list would grow without bounds, and you'd be essentially leaking memory again. If there aren't going to be any quiescent periods, you need to find an alternative mechanism for reclaiming the nodes. The key is to identify when no more threads are accessing a particular node so that it can be reclaimed. By far the easiest such mechanism to reason about is the use of *hazard pointers*.

Figure 7.1. Three threads call `pop()` concurrently, showing why you must check `threads_in_pop` after claiming the nodes to be deleted in `try_reclaim()`.